

20-ma'ruza. Rekursiv va qayta yuklanuvchi metodlar

Reja:

1. Rekursiya
2. Qayta yuklanuvchi funksiyalar
3. Rekursiv funksiyalarga oid masalalar

C# dasturlash tili metodga metodning o'zini chaqirish imkonini beradi. Bu jarayon rekursiya deb ataladi va o'zini chaqiradigan metod rekursiv deb ataladi. Umuman olganda, rekursiya biror narsaning o'zini-o'zi belgilaydigan jarayondir. Shu nuqtayi nazardan, u bir oz siklik jarayon ta'rifini eslatadi. Rekursiv metod, asosan, u metod o'zini chaqiradigan kodni o'z ichiga olganligi bilan farq qiladi. Rekursiya dasturni boshqarishning eng samarali mexanizmlaridan biridir.

Rekursiya deb metod tanasida shu metodning o'zini chaqirishiga aytiladi. Rekursiya ikki xil bo'ladi:

oddiy - agar metod o'z tanasida o'zini chaqirsa;

vositali - agar birinchi metod ikkinchi metodni chaqirsa, ikkinchisi esa o'z navbatida birinchi metodni chaqirsa.

Odatda rekursiya matematikada keng qo'llaniladi. Chunki, aksariyat matematik formulalar rekursiv aniqlanadi. Misol tariqasida faktorialni hisoblash formulasini

$$n! = \begin{cases} 1, & \text{agar } n = 0; \\ n \cdot (n-1)!, & \text{agar } n > 0, \end{cases}$$

va sonning butun darajasini hisoblashni ko'rishimiz mumkin:

$$x^n = \begin{cases} 1, & \text{agar } n = 0; \\ x * x^{n-1}, & \text{agar } n > 0. \end{cases}$$

Ko'rinib turibdiki, navbatdagi qiymatni hisoblash uchun funksiyaning «oldingi qiymati» ma'lum bo'lishi kerak. C# tilida rekursiya matematikadagi rekursiyaga o'xshash.

Rekursiyani tushuntirish uchun sonning faktorialini hisoblash algoritmini ko'rib chiqamiz. N butun sonining faktoriali 1 dan N gacha bo'lgan barcha butun sonlarning ko'paytmasidir. Masalan, 3 ning faktoriali $1 \times 2 \times 3$ yoki 6 ga teng. Quyidagi dastur misolida sonning faktorialini hisoblashning rekursiv metodi ko'rsatilgan. Taqqoslash uchun ushbu dasturda sonning faktorialini hisoblashning rekursiv bo'lmagan versiyasi ham mavjud.

```
using System;
class Factorial
{
    // Rekursiv metod.
    public int FactR(int n)
    {
        int result;
        if (n == 0 || n == 1) return 1;
        result = FactR(n - 1) * n;
        return result;
    }
    // Iteratsion metod.
    public int FactSikl(int n)
    {
        int t, result;
        result = 1;
        for (t = 1; t <= n; t++)
            result *= t;
        return result;
    }
}
```

```

}
using System;
namespace rekursiya
{
    class Program
    {
        public static void Main(string[] args)
        {
            Factorial f = new Factorial();
            Console.WriteLine("Rekursiv usul bilan hisoblangan faktoriallar");
            Console.WriteLine("3 ning faktoriali " + f.FactR(3));
            Console.WriteLine("4 ning faktoriali " + f.FactR(4));
            Console.WriteLine("5 ning faktoriali " + f.FactR(5));
            Console.WriteLine();
            Console.WriteLine("Iteratsiya bilan hisoblangan faktoriallar");
            Console.WriteLine("3 ning faktoriali " + f.FactSikl(3));
            Console.WriteLine("4 ning faktoriali " + f.FactSikl(4));
            Console.WriteLine("5 ning faktoriali " + f.FactSikl(5));
            Console.ReadKey(true);
        }
    }
}

```

Ushbu dastur bajarilganda quyidagi natija olinadi

Rekursiv usul bilan hisoblangan faktoriallar

3 ning faktoriali 6

4 ning faktoriali 24

5 ning faktoriali 120

Sikl bilan hisoblangan faktoriallar

3 ning faktoriali 6

4 ning faktoriali 24

5 ning faktoriali 120

Rekursiv bo'lmagan FactSikl() metodining ishlash prinsipi juda aniq. U 1 dan boshlanadigan raqamlar ketma-ket bir-biriga ko'paytirilib, asta-sekin faktorial beradigan natijani hosil qiladigan sikldan foydalanadi.

Agar FaktR() metodiga $n > 0$ qiymat berilsa, quyidagi holat ro'y beradi: shart operatorining else bo'limidagi qiymati(n qiymati) stekda eslab qolinadi. Hozircha qiymati noma'lum $n-1$ faktorialni hisoblash uchun shu metodning o'zi $n-1$ qiymati bilan bilan chaqiriladi. O'z navbatida, bu qiymat ham eslab qolinadi (stekka joylanadi) va yana metod chaqiriladi va hakoza. Metod $n=0$ qiymat bilan chaqirilganda if operatorining sharti ($n==1$) rost bo'ladi va «return 1;» amali bajarilib, ayni shu chaqirish bo'yicha 1 qiymati qaytariladi. Shundan keyin «teskari» jarayon boshlanadi - stekda saqlangan qiymatlar ketma-ket olinadi va ko'paytiriladi: oxirgi qiymat - aniqlangandan keyin (1), u undan oldingi saqlangan qiymatga 1 qiymatiga ko'paytirib FactR(1) qiymati hisoblanadi, bu qiymat 2 qiymatiga ko'paytirish bilan FactR(2) topiladi va hokaza. Jarayon FactR(n) qiymatini hisoblashgacha «ko'tarilib» boradi. Bu jarayonni, $n=3$ uchun faktorial hisoblash sxemasini 5.3.1-rasmda ko'rish mumkin:

↓	FactR(3) = 3*FactR(2)	↓	FactR(3) = 3*FactR(2)	↓	FactR(3) = 3*FactR(2)	↑	FactR(3) = 3*FactR(2)
↓	FactR(2) = 2*FactR(1)	↓	FactR(2) = 2*FactR(1)	↑	FactR(2) = 2*FactR(1)		

↓	FactR(1) = 1*FactR(0)	↑	FactR(1) = 1*FactR(0)
↑	FactR(0) = 1		

5.3.1-rasm. 3! ni hisoblash sxemasi

Rekursiv metodlarni to‘g‘ri ishlashi uchun rekursiv chaqirishlarning to‘xtash sharti bo‘lishi kerak. Aks holda rekursiya to‘xtamasligi va o‘z navbatida metod ishi tugamasligi mumkin. Faktorial hisoblashida rekursiv tushishlarning to‘xtash sharti metod parametri $n=0$ bo‘lishidir (shart operatorining rost shoxi).

Har bir rekursiv murojaat qo‘shimcha xotira talab qiladi - metodlarning lokal obyektlari (o‘zgaruvchilari) uchun har bir murojaatda stekdan yangidan joy ajratiladi. Masalan, rekursiv metodga 100 marta murojaat bo‘lsa, jami 100 lokal obyektlarning majmuasi uchun joy ajratiladi. Ayrim hollarda, ya‘ni rekursiyalar soni yetarlicha katta bo‘lganda, stek o‘lchami cheklanganligi sababli (real rejimda 64Kb o‘lchamgacha) u to‘lib ketishi mumkin. Bu holatda programma o‘z ishini «Stek to‘lib ketdi» xabari bilan to‘xtadi.

Metod o‘zini chaqirganda, xotira yangi lokal o‘zgaruvchilar va parametrlar uchun tizim stekiga ajratiladi va metod kod boshidan bu yangi o‘zgaruvchilar va parametrlar bilan bajariladi. Metod rekursiv chaqirilganda uning yangi nusxasi yaratilmaydi, faqat uning yangi argumentlaridan foydalaniladi. Har bir rekursiv murojaatdan qaytgandan so‘ng eski lokal o‘zgaruvchilar va parametrlar stekdan chiqariladi va bajarilish metodda chaqiruv nuqtasidan davom etadi. Rekursiv metodlarni prinsipial jihatdan asta-sekin siqilib, keyin to‘g‘rilanadigan kamon bilan solishtirish mumkin.

Quyida belgilar satrini teskari tartibda chiqarish uchun rekursiyaning yana bir misoli keltirilgan. Ushbu satr rekursiv DisplayRev() metodiga argument sifatida berilgan.

// Rekursiya yordamida belgilar qatorini teskari tartibda chiqarish.

```
using System;
class RevStr
{
    // Belgilar satrini teskari tartibda chiqarish metodi
    public void DisplayRev(string str)
    {
        if (str.Length > 0)
            DisplayRev(str.Substring(1, str.Length - 1));
        else
            return;
        Console.Write(str[0]);
    }
}
class RevStrDemo
{
    static void Main()
    {
        string s = "Bu test rejimi";
        RevStr rsOb = new RevStr();
        Console.WriteLine("Kiritilgan satr: " + s);
        Console.Write("Teskari satr: ");
        rsOb.DisplayRev(s);
        Console.WriteLine();
        Console.ReadKey();
    }
}
```

Dastur bajarilish natijasida:

Kiritilgan satr: Bu test rejimi

Teskari satr: imijer tset uB

DisplayRev() metodi har doim chaqirilganda u str argumenti bilan ifodalangan belgilar satrining uzunligini tekshiradi. Agar satr uzunligi nolga teng bo'lsa, u holda DisplayRev() metodi berilgan satrdan bir belgi kam bo'lgan yangi satr bilan rekursiv chaqiriladi. Bu metodga nol uzunlikdagi satr uzatilgunga qadar bu jarayon takrorlanadi. Shundan so'ng, DisplayRev() metodiga barcha rekursiv murojaatlar teskari tartibda ochiladi. Har bir bunday chaqiruvdan so'ng str argumenti bilan ifodalangan satrning birinchi belgisi chop etiladi va natijada butun satr teskari tartibda chop etiladi.

Ko'pgina metodlarning rekursiv versiyalari takroriy murojaatlar uchun tizim resurslari tufayli ularning iterativ ekvivalentlariga qaraganda biroz sekinroq ishlashi mumkin. Agar bunday murojaatlar juda ko'p bo'lsa, tizim to'lib ketishi mumkin. Rekursiv metodning parametrlari va lokal o'zgaruvchilari tizim stekida saqlanganligi va bu metodga har bir yangi chaqiruv, ularning yangi nusxasini yaratganligi sababli bir nuqtada stek tugashi mumkin. Bunday holda, istisno yuzaga keladi va umumiy til ish vaqti (CLR) tegishli istisnoni chiqaradi. **Rekursiyaning asosiy afzalligi** shundaki, u ba'zi algoritmlarni iterativ usuldan ko'ra aniqroq va sodda tarzda amalga oshirish imkonini beradi. Masalan, tezkor saralash algoritmini iterativ tarzda amalga oshirish juda qiyin. Rekursiv metodlarni yozayotganda, metod rekursiyasiz qaytishini ta'minlash uchun shartli kod qismini, masalan, agar tegishli joyga qo'shish zarur. Aks holda, bir marta chaqirilgan rekursiv metod umuman qaytmasligi mumkin. Bunday xatolik dasturlash amaliyotida rekursiyani amalga oshirish uchun juda xosdir. Bunday holda, rekursiv metodda nima sodir bo'layotganini kuzatish va xatolik yuzaga kelsa, uning bajarilishini to'xtatish uchun WriteLine() metodidan foydalanish tavsiya etiladi.

Ko'pincha misol sifatida faktorial yordamida rekursiyalarni o'rgatilganligini ko'rish mumkin, xuddi yuqoridagi misoldagidek. Ammo dasturchilar faktorialni haqiqatan ham rekursiya orqali hisoblashga o'rganib qolishi mumkin. Faktorialni rekursiya bilan hisoblash mumkin, lekin buni oddiy sikl orqali bajarish ancha yaxshi, tezroq va samaraliroqdir.

Rekursiyani o'rgatish uchun tez tartiblash algoritmini keltirish mumkin. Algoritm massivni ma'lum bir atributga ko'ra taqsimlashga asoslanadi va massivning har bir yarmi uchun saralashni yana chaqiradi. Ikkinchi bosqichda metod yana ikki qismga bo'lingan kichikroq massivni oladi. Shunday qilib, har bir bosqichda massiv kichikroq qismlarga bo'linadi. Ushbu algoritmni rekursiya orqali ko'rib chiqaylik:

```
using System;
namespace Rekursiya
{
    class Program
    {
        static int[] array = { 7, 75, 28, 47, 31, 105, 17, 9, 27, 119, 57 };
        public static void Main(string[] args)
        {
            sort(0, array.Length - 1);
            foreach (int i in array)
                Console.WriteLine(i);
            Console.ReadLine();
            Console.ReadKey(true);
        }
        static void sort(int l, int r)
        {
            int i = l;
            int j = r;
            int x = array[(l + r) / 2];
            do
```

```

    {
        while (array[i] < x) i++;
        while (array[j] > x) j--;
        if (i <= j)
        {
            int y = array[i];
            array[i] = array[j];
            array[j] = y;
            i++;
            j--;
        }
    } while (i < j);

    if (l < j)
        sort(l, j);
    if (l < r)
        sort(i, r);
}
}
}

```

Ushbu dastur matnini o‘zingiz tahlil qilib ko‘ring. Koddagi rekursiv jarayon eng oxirida bo‘lib, sort() metodi o‘zini chaqiradi va massiv indekslarini uzatadi.

Metodlarni qayta yuklash

C# bir xil sinfnining ikki yoki undan ortiq metodlariga, parametrlari boshqacha e‘lon qilingan bo‘lsa, bir xil nomga ega bo‘lish imkonini beradi. Bunda qayta yuklanuvchi jarayonning o‘zi qayta yuklanuvchi metodlar deyiladi. Umuman olganda, metodni qayta yuklash uchun uning turli xil variantlarini e‘lon qilish kifoya, qolganlari bilan kompilyator shug‘ullanadi. Ammo shu bilan birga, quyidagi muhim shartga rioya qilish kerak: har bir metod uchun parametrlarning turi yoki soni boshqacha bo‘lishi kerak. Ikki metodning faqat qaytarish turlari bo‘yicha farqlanishi yetarli emas. Ular, shuningdek, parametrlarining turlari yoki soni bo‘yicha farq qilishi kerak. (Bunday holatlarda, qaytarish turlari C# kompilyatoriga qaysi metoddan foydalanishni hal qilish uchun yetarli ma‘lumot bermaydi.) Qayta yuklangan metodlar qaytarish turlarida ham farq qilishi mumkin. Qayta yuklangan metod chaqirilganda, o‘tkazilgan argumentlarga parametrlari mos keladigan (turi va soni bo‘yicha) metodning varianti bajariladi.

Quyida qayta yuklash metodini ko‘rsatish uchun oddiy misol keltirilgan.

```

using System;
class QaytaYuklash
{
    public void Overload()
    {
        Console.WriteLine("Parametrsiz");
    }
    // Overload metodini bitta int parametri bilan qayta yuklash.
    public void Overload(int a)
    {
        Console.WriteLine("Bitta parametr: " + a);
    }
    // Overload metodini ikkita int parametri bilan qayta yuklash.
    public int Overload(int a, int b)
    {
        Console.WriteLine("Ikkita int parametr: " + a + " " + b);
        return a + b;
    }
}

```

```

    }
    // Overload metodini ikkita double parametri bilan qayta yuklash.
    public double Overload(double a, double b)
    {
        Console.WriteLine("Ikkita double parametr: " + a + " " + b);
        return a + b;
    }
}
class QaytaYuklashDemo
{
    static void Main()
    {
        QaytaYuklash ob = new QaytaYuklash();
        int resI;
        double resD;
        // Overload() metodining barcha variantlarini chaqiring.
        ob.Overload();
        Console.WriteLine();
        ob.Overload(2);
        Console.WriteLine();
        resI = ob.Overload(4, 6);
        Console.WriteLine("ob.Overload(4, 6) metodini chaqirish natijasi: " + resI);
        Console.WriteLine();
        resD = ob.Overload(1.1, 2.32);
        Console.WriteLine("ob.Overload(1.1, 2.32) metodini chaqirish natijasi: " + resD);
        Console.ReadKey();
    }
}

```

Dastur bajarilishi natijasida:

Paramsiz

Bitta parametr: 2

Ikkita int parametr: 4 6

ob.Overload(4, 6) metodini chaqirish natijasi: 10

Ikkita double parametr: 1,1 2,32

ob.Overload(1.1, 2.32) metodini chaqirish natijasi: 3,42

Ko'rib turganingizdek, Overload() metodi to'rt marta qayta yuklangan. Birinchi variant hech qanday parametрни olmaydi, ikkinchisi bitta butun son parametrini, uchinchisi ikkita butun son parametrini, to'rtinchisi esa ikkita juft parametрни oladi. Overload() metodining dastlabki ikkita varianti void turda bo'lganligi uchun metod hech qanday qiymat qaytarmaydi, qolgan ikkitasi esa ma'lum qiymatni qaytaradi. Qaytarish turi metodni qayta yuklash uchun hech qanday rol o'ynamaydi. Shuning uchun quyidagi kod qismida Overload() metodining ikki xil (qaytish turi bo'yicha) variantidan foydalanishga urinish xatolikka olib keladi.

```

public void Overload(int a)
{
    Console.WriteLine("Bitta parametr: " + a);
}
/* Xato! Ikkita Overload(int) metodi e'lon qilishga ruxsat berilmaydi,
ular har xil turdagi qiymatlarni qaytarsa ham. */
public int Overload(int a)
{
    Console.WriteLine("Bitta parametr: " + a);
}

```

```

    return a * a;
}

```

Yuqoridagi kodga berilgan izohlardan ko‘rinib turibdiki, Overload() metodining ikkala ko‘rinishida qiymat qaytarish turlaridagi farqlar ushbu metodni qayta yuklash uchun yetarli emas. C# bir qator yashirin (ya’ni avtomatik) turdagi imkoniyatlarni taqdim etadi. Ushbu imkoniyatlar qayta yuklangan metodlarning parametrlariga ham tegishli. Misol tariqasida quyidagi dastur kodini ko‘rib chiqamiz.

```

using System;
class Overload2
{
    public void MyMeth(int x)
    {
        Console.WriteLine("MyMeth(int) metodida:" + x);
    }
    public void MyMeth(double x)
    {
        Console.WriteLine("MyMeth(double) metodida: " + x);
    }
}
class TypeConv
{
    static void Main()
    {
        Overload2 ob = new Overload2();
        int i = 10;
        double d = 10.1;
        byte b = 99;
        short s = 10;
        float f = 11.5F;
        ob.MyMeth(i); // ob.MyMeth(int) metodini chaqirish
        ob.MyMeth(d); // ob.MyMeth(double) metodini chaqirish
        ob.MyMeth(b); // ob.MyMeth(int) metodini turini o‘zgartirib chaqirish
        ob.MyMeth(s); // ob.MyMeth(int) metodini turini o‘zgartirib chaqirish
        ob.MyMeth(f); // ob.MyMeth(int) metodini turini o‘zgartirib chaqirish
    }
}

```

Dastur bajarilishi natijasida:

```

MyMeth(int) metodida: 10
MyMeth(double) metodida: 10,1
MyMeth(int) metodida: 99
MyMeth(int) metodida: 10
MyMeth(double) metodida: 11,5

```

Ushbu misolda MyMeth() metodining faqat ikkita varianti yozilgan: int turidagi parametr bilan va double turli parametr bilan. Biroq MyMeth() metodiga bayt, short yoki float qiymatini kiritish mumkin. Shunday qilib, agar bu metodga bayt yoki short turidagi qiymat uzatilsa, u holda C# kompilyatori bu qiymatni avtomatik ravishda int turiga o‘zgartiradi va natijada bu metodning MyMeth(int) varianti chaqiriladi. Agar unga float turidagi qiymat uzatilsa, u double turiga aylantiriladi va natijada bu usulning MyMeth(double) varianti chaqiriladi. Shu bilan birga yopiq turdagi konvertatsiyalar faqat parametr va argument turlari o‘rtasida aniq moslik bo‘lmasa amalga oshiriladi. Masalan, quyida bayt parametrini belgilaydigan MyMeth() metodining variantini qo‘shadigan oldingi dasturning biroz o‘zgartirilgan versiyasi keltirilgan.

```

using System;

```

```

class Overload2
{
    public void MyMeth(int x)
    {
        Console.WriteLine("MyMeth(int) metodida: " + x);
    }
    public void MyMeth(byte x)
    {
        Console.WriteLine("MyMeth(byte) metodida: " + x);
    }
    public void MyMeth(double x)
    {
        Console.WriteLine("MyMeth(double) metodida: " + x);
    }
}
class TypeConv
{
    static void Main()
    {
        Overload2 ob = new Overload2();
        int i = 10;
        double d = 10.1;
        byte b = 99;
        short s = 10;
        float f = 11.5F;
        ob.MyMeth(i); // ob.MyMeth(int) metodini chaqirish
        ob.MyMeth(d); // ob.MyMeth(double) metodini chaqirish
        ob.MyMeth(b); // ob.MyMeth(int) metodini turini o'zgartirib chaqirish
        ob.MyMeth(s); // ob.MyMeth(int) metodini turini o'zgartirib chaqirish
        ob.MyMeth(f); // ob.MyMeth(int) metodini turini o'zgartirib chaqirish
        Console.ReadKey();
    }
}

```

Dastur natijasida quyidagi natija qaytariladi:

MyMeth(int) metodida: 10
 MyMeth(double) metodida: 10,1
 MyMeth(byte) metodida: 99
 MyMeth(int) metodida: 10
 MyMeth(double) metodida: 11,5